Final SAST and DAST Security Testing Report

Proprietary Statement

(This document contains information from TestingXperts that is highly confidential and privileged. The information is intended for the personal use of TestingXperts)

TestingXperts
28/04/2026

Document Details:

Company	TestingXperts
Document Title	Tx-Catalyst Final SAST and DAST Report
Submission Date	28-May-2026
Classification	Highly Confidential
Document Type	Report

Document History:

Version	Date	Author(s)	Comments
1.0	28th May'26	Security Team	Final SAST and DAST Report

Contents

1. Executive Summary

"TestingXperts" conducted a Static and Dynamic Application Security Testing assessment against their application "Tx_Catalyst". The objective of the test was to meet information security standards like OWASP and NIST and to simulate real-world attack scenarios and aims to exploit vulnerabilities, identify design flaws, leverage combination of lower impact flaws into higher impact vulnerabilities, and determine if identified flaws affect the confidentiality, integrity, or availability of the application.

Following target URL were in scope: -

http://192.168.3.90:3002

As part of this Testing, we tried to capture the business logic flow of the application, list down the potential threats to the application, and prepare this Security test report.

Our Approach was to meet Information Security Industry standards like OWASP Top 10, and NIST.

80% – 85% attacks in the real world are based on these techniques. Additionally, we make an effort to perform attacks relevant to business logic.

During this engagement, we primarily focused on **Authentication and Authorization mechanism, Session Management, Input Validation, Security Misconfiguration, Cryptography, Business Logic bypasses, Server and Client-side attacks, etc.**

1.1 Test Timeline

Security testing on web application was performed during VAPT engagement on 28th May 2026.

1.2 Approach

Activities
Application Understanding, Information Gathering and Identifying Application Entry Points
Understanding of the Source Code and Dependencies
Vulnerability Scanning and Manual Verification
Manual Penetration Testing as per OWASP and NIST Methodologies
Vulnerability Analysis
Removal of False Positive
Report Preparation

1.3 Activities

The following set of security testing activities are performed by the team:

- Understanding of the Source Code and Dependencies used for the platform.
- Understanding of the applications, security requirements, security assets.
- Understanding of user roles, access levels, authentications and Business Logic.
- Vulnerability Scanning and Penetration Testing as per OWASP 10 and OSSTMM standards using TX's customized security testing framework (based on open-source tools and Burp Suite Pro).
- Determine possible extent access or impact to the system by attempting to exploit vulnerabilities.
- Vulnerability Analysis to remove 'false positives' and identify potential remedies.
- Provide tactical recommendations to address security issues of immediate consequence.
- Provide strategic recommendations to enhance security by leveraging industry best practice

1.4 Summary of Findings

TestingXperts Security Team discovered **8** vulnerabilities during the course of SAST and DAST analysis. The report classifies each vulnerability in appropriate categories, and we have suggested mitigation strategy for them.

Going ahead, TestingXperts Security Team recommends:

1. Address and remediate the vulnerabilities identified through SAST in accordance with secure coding best practices across the development lifecycle.
2. Fixing of reported vulnerabilities.

3. Follow the Iterative Secure SDLC process, this includes complete security regression test with any new component that is added.

Vulnerabilities By Severity

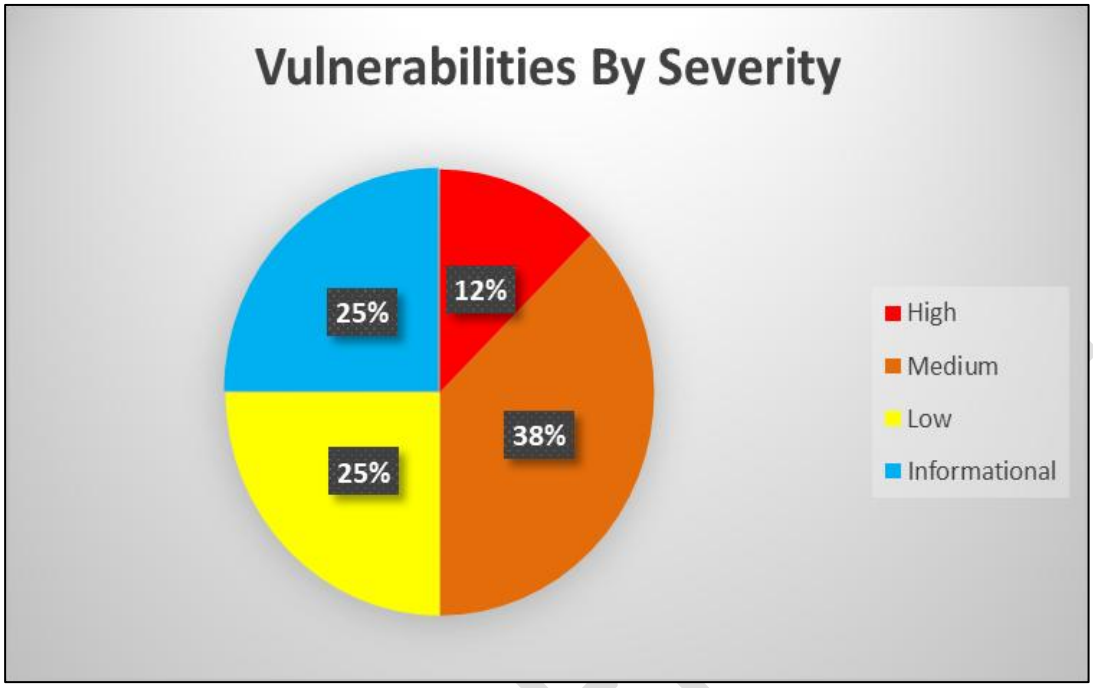
The following vulnerabilities were identified during the engagement: -

Sr. No.	Vulnerability Name	Severity	CVSS Score
1.	Improper Authorization- Insufficient Access Control	High	7.3
2.	Encryption Not Enforced	Medium	6.5
3.	No HTTP Strict Transport Security (HSTS)	Medium	4.2
4.	Cross-Frame Scripting (XFS) / Clickjacking Possible	Medium	4.2
5.	Missing Cookie 'Flags'	Low	3.1
6.	Missing Security Headers	Low	3.1
7.	Lack of Post-Quantum Cryptography (PQC) Support	Informational	0.0
8.	Weak Cryptography	Informational	0.0

Tabular Summary

Total Number of Vulnerabilities	8
Critical Severity Vulnerabilities	0
High Severity Vulnerabilities	1
Medium Severity Vulnerabilities	3
Low Vulnerabilities	2
Informational Vulnerabilities	2

Classification of Vulnerabilities



"As pie chart indicates that 12% vulnerabilities are identified as High, 38% vulnerabilities are identified as Medium ,25% as Low and 25% vulnerabilities are acknowledged as Informational."

2. Vulnerabilities Description

High Severity Vulnerabilities

2.1 Improper Authorization - Insufficient Access Control

Vulnerability Information:

The application utilizes user roles with distinct privilege levels, but does not properly check that a user is authorized to access / modify a resource or perform some action before allowing them to do so.

Exploit Description:

An attacker with a valid low-privileged user account may attempt to access restricted functionalities by modifying URL paths to include administrative endpoints. Such endpoints may be identified through techniques like crawling, enumeration, or predictable naming conventions. In the absence of proper server-side authorization checks, the application may fail to verify whether the requesting user possesses the required roles (e.g., Admin or Executive) to access these internal routes. As a result, unauthorized users may gain access to sensitive data, administrative functionalities, or critical system configurations.

Severity Description:

A low privileged user can access / modify resources and perform actions to which they do not have sufficient privileges. This could lead to unauthorized disclosure or modification of sensitive information, as well as privilege escalation.

Vulnerability Severity:

- **Risk -** High
- **Impact -** High
- **Exploitability -** High
- **CVSS Score: -** CVSS v3.1 Vector: AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N = 7.3 **(High)**

Category: Authorization

Instance:

http://192.168.3.90:3002/

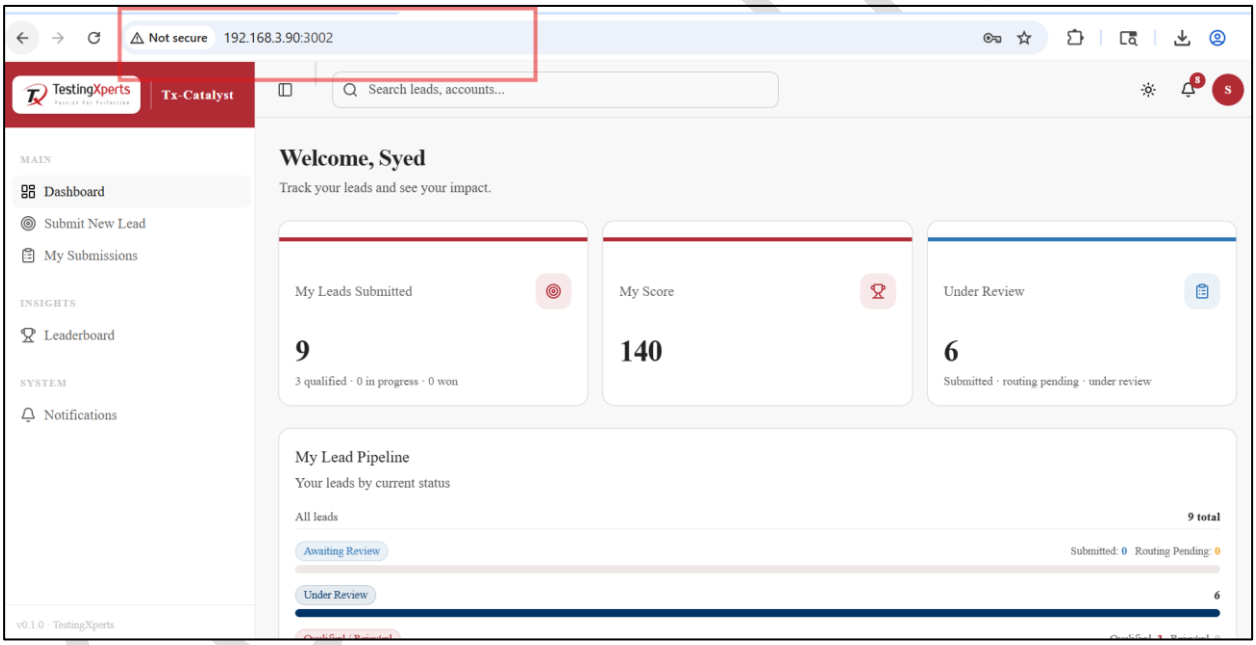
Proof-Of-Concept:

Provided are the request/response and screenshot which demonstrate that the application lacks proper authorization controls, allowing unauthorized access to restricted resources.

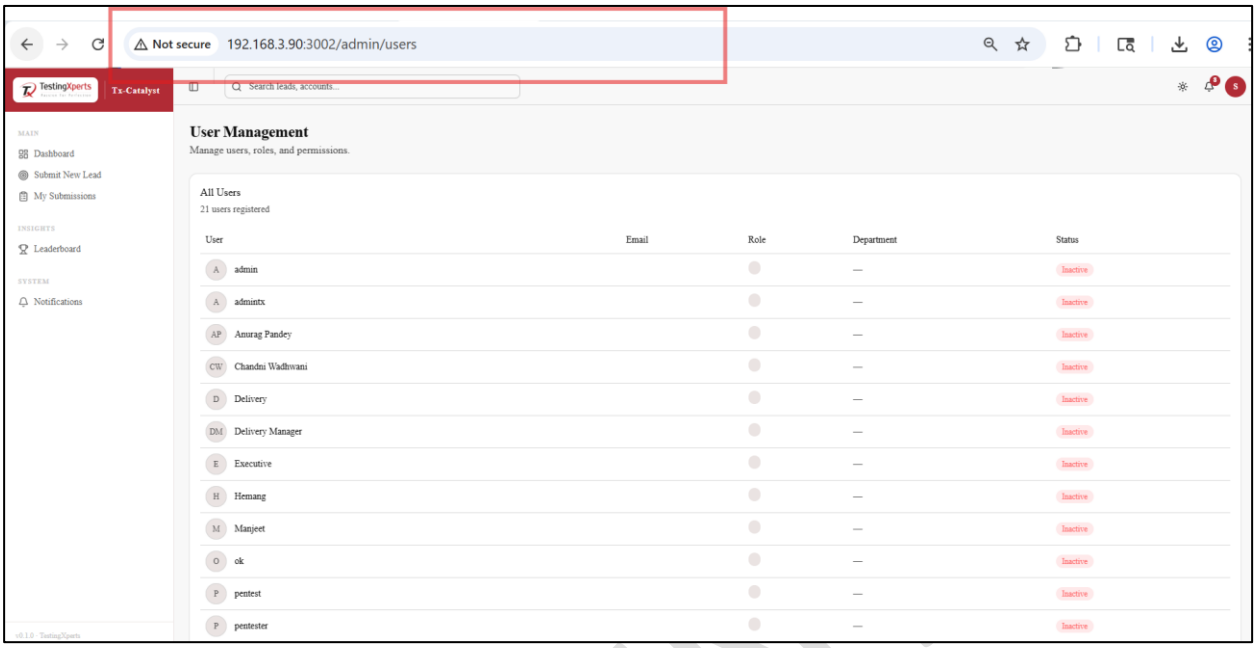
Steps to Reproduce:

1. Log in to the application using a Normal User account.
2. Observe the limited sidebar menu (e.g., Dashboard, Submit New Lead, My Submissions). (See Screenshot 1)
3. Manually change the URL in the browser address bar to http://192.168.3.90:3002/admin/users.
4. Observe that the application loads the User Management page, displaying all registered users and their roles, despite the user not having administrative privileges. (See Screenshot 2)
5. Similarly, navigate to http://192.168.3.90:3002/admin/stakeholder-mapping to access routing configurations. (See Screenshot 3)
6. A user can manipulate the URL to access the Accounts module. Upon modifying the endpoint, the Accounts section is exposed, allowing the user to create new accounts despite lacking the necessary privileges. (See Screenshot 4)

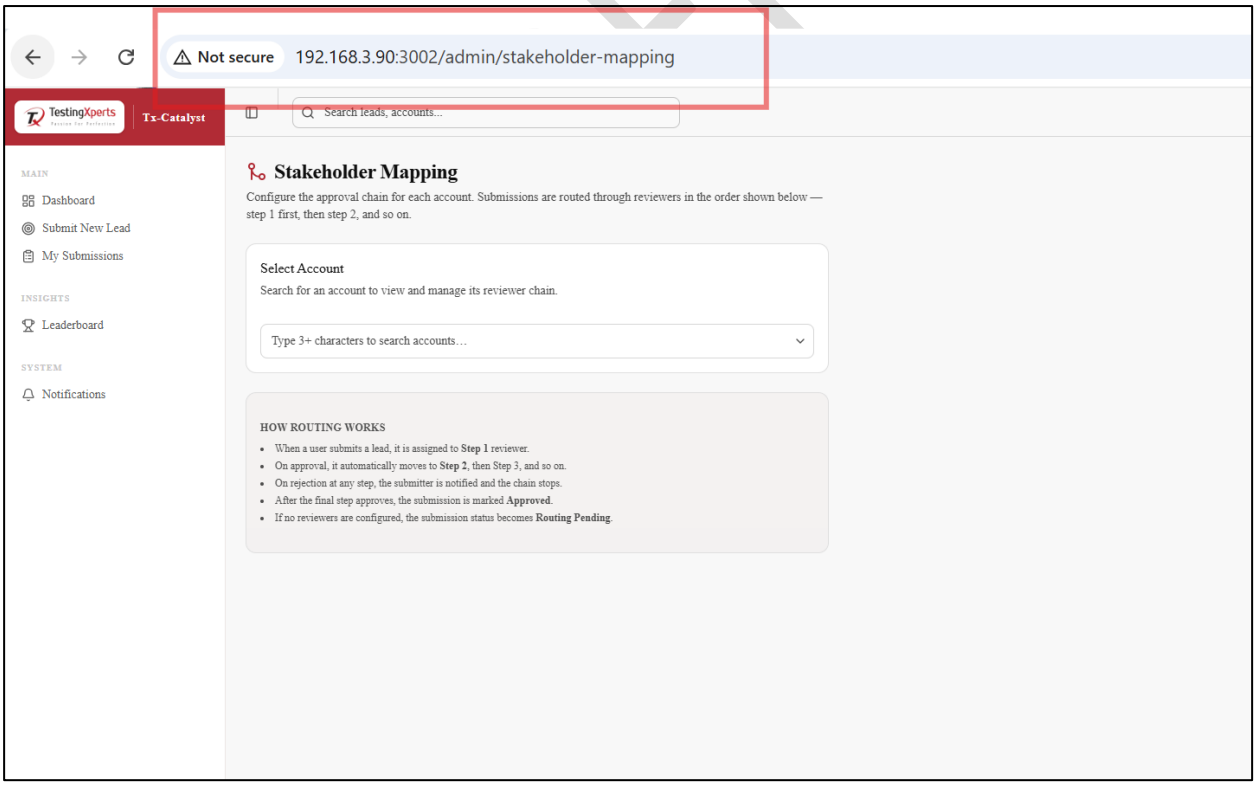
Screenshot 1:



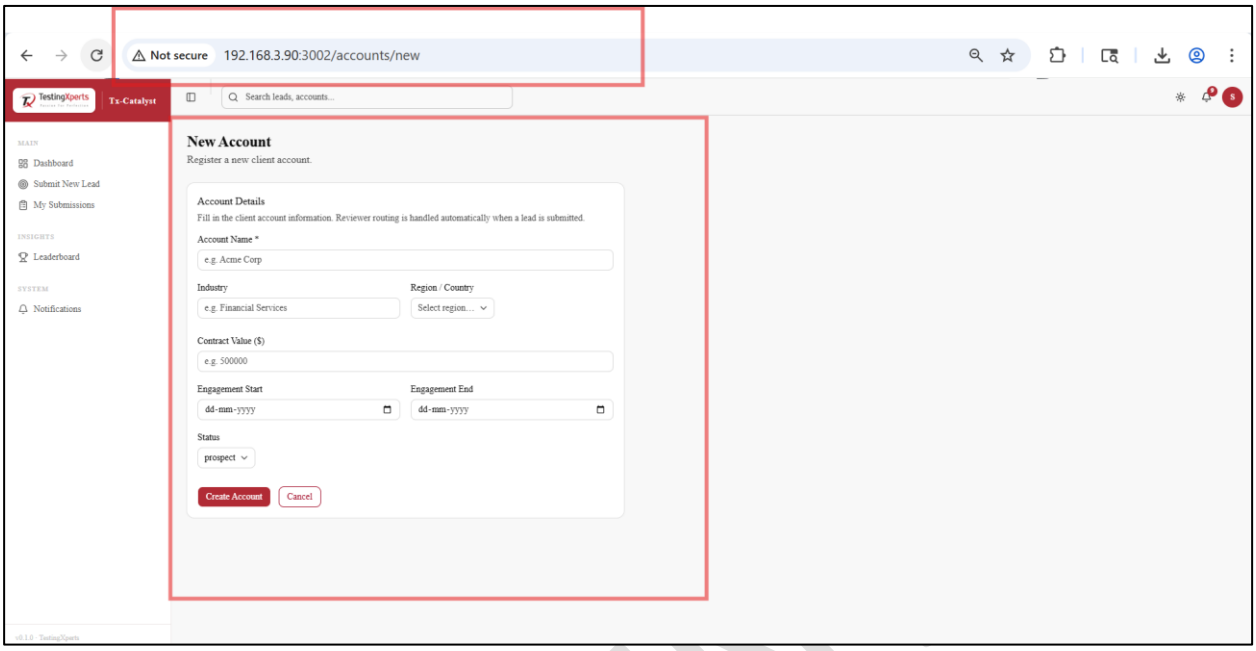
Screenshot 2:



Screenshot 3:



Screenshot 4:



Remediation:

If a user attempts to access / modify a resource or perform some action, the application should first verify that they have sufficient privilege levels to do so, and reject their request if they are underprivileged. This access control check should also verify that the user has any native access rights to the resource within the application (i.e. a user should not be able to access resources owned by another user unless this is purposefully allowed by the application).

Medium Severity Vulnerabilities

2.2 Encryption Not Enforced

Vulnerability Information:

During the application test, it was detected that the site uses an encrypted connection to protect sensitive information. However, it was possible to receive these resources using HTTP, which means that sensitive information may be sent unencrypted to the server and/or back to the user.

Exploit Description:

An attacker attempting to monitor or manipulate transmitted data would seek to actively reroute data traffic or passively observe clear-text communications. A number of techniques could be employed to accomplish this including ARP spoofing, DNS spoofing, route manipulation, and switch monitoring. The ease of exploit is generally related to the proximity of the attacker to the victim.

Severity Description:

All application data including user credentials could be manipulated and observed by an attacker. User credentials are often reused across disparate systems which extends the reach of the compromise. The confidentiality and integrity of application data could be undermined.

Vulnerability Severity:

- **Risk -** Medium
- **Impact -** Medium
- **Exploitability -** Low
- **CVSS Score: -** CVSS v3.1 Vector: AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:N = 6.5 **(Medium)**

Category: Configuration

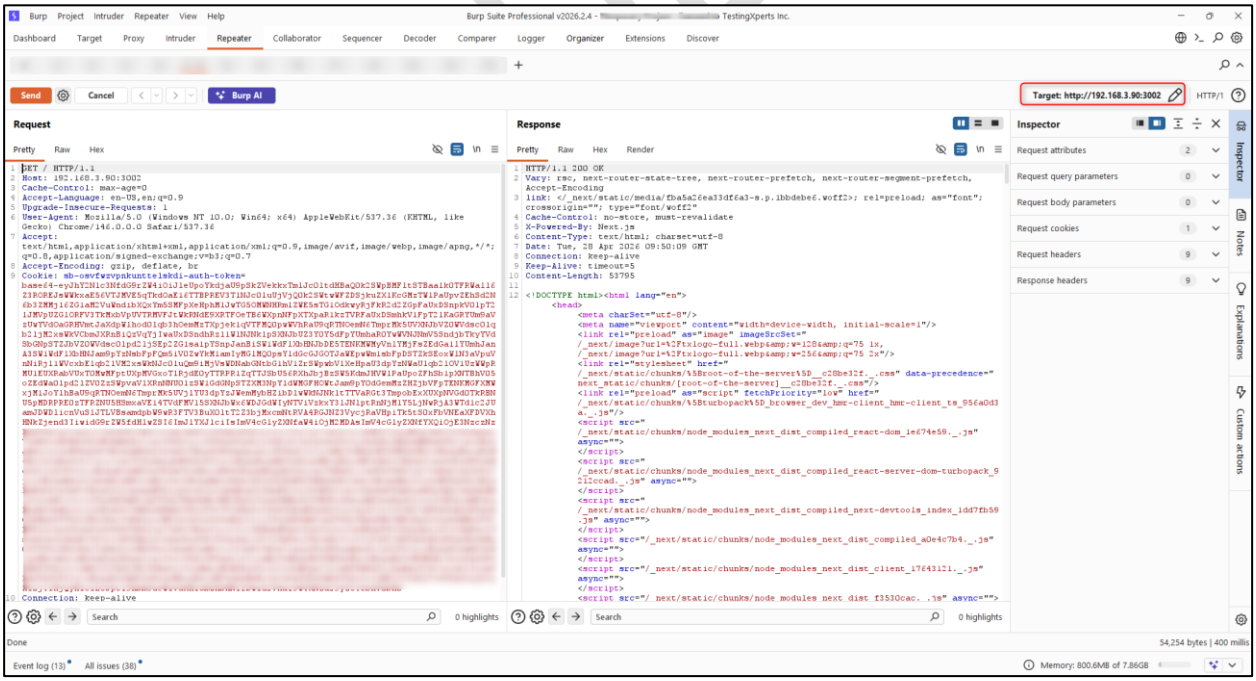
Instance:

http://192.168.3.90:3002/

Proof-Of-Concept:

Provided is the screenshot which demonstrate that the application does not enforce encryption.

Screenshot:



Remediation:

Applications should use transport-level encryption (SSL/TLS) to protect all communications passing between the client and the server. The Strict-Transport-Security HTTP header should be used to ensure that clients refuse to access the server over an insecure connection.

2.3 No HTTP Strict Transport Security (HSTS)

Vulnerability Information:

The application does not use Strict Transport Security, which is an additional security feature that ensures SSL/TLS is used, and that no sensitive information is sent via HTTP only. This feature is recognized by most browsers.

Exploit Description:

During testing it was observed that the Strict-Transport-Security header is not utilized and therefore not set as a condition. Attackers may utilize hacking packages such as SSLStrip to host versions of the vulnerable application without the 'https' protocol in place between the user and the server - allowing a 'man in the middle' attack.

Severity Description:

Without Strict-Transport-Security, an attacker may trick a user into submitting non-SSL/TLS requests and observe the requests in order to hijack the session by observing cookie values, usernames, and passwords. This may allow an attacker to compromise a user's session or account or maliciously modify the way the website appears to the user. The collateral damage to exploitation of this vulnerability can be severe and include the compromise of administrative accounts and extensive reputation damage.

Vulnerability Severity:

- **Risk -** Medium
- **Impact -** Low
- **Exploitability –** Low
- **CVSS Score: -** CVSS v3.1 Vector: AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N = 4.2 **(Medium)**

Category: Configuration

Instance:

http://192.168.3.90:3002/

Proof-Of-Concept:

Provided are the request and response demonstrating that HSTS is not enabled on the application.

Vulnerable Request/Response:

Request:

```
GET / HTTP/1.1
Host: 192.168.3.90:3002
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/web
p,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
[Redacted for Brevity]
```

Response:

```
HTTP/1.1 200 OK
Vary: rsc, next-router-state-tree, next-router-prefetch, next-router-
segment-prefetch, Accept-Encoding
link: </_next/static/media/fba5a26ea33df6a3-s.p.1bbdebe6.woff2>;
rel=preload; as="font"; crossorigin=""; type="font/woff2"
Cache-Control: no-store, must-revalidate
X-Powered-By: Next.js
Content-Type: text/html; charset=utf-8
Date: Tue, 28 Apr 2026 09:50:09 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 53795
[Redacted for Brevity]
```

Remediation:

Applications should use transport-level encryption (SSL/TLS) to protect all communications passing between the client and the server. The Strict-Transport-Security HTTP header should be used to ensure that clients refuse to access the server over an insecure connection. An example header is: Strict-Transport-Security: max-age=31536000; includeSubDomains Additional information: https://www.owasp.org/index.php/HTTP_Strict_Transport_Security_Cheat_Sheet.

2.4 Cross-Frame Scripting (XFS) / Clickjacking Possible

Vulnerability Information:

The application is susceptible to malicious framing attacks, because it does not adequately prevent itself from being encapsulated with an external frame.

Exploit Description:

This attack revolves around a malicious person creating a web page with an iframe on a site within their control (EvilSite.com). The attacker's site will include on their page a

visible iframe displaying within it the vulnerable application (Example.com). This is typically a login, forgot password, or some type of registration page. The attacker can hide the frame's borders and expand the frame to cover the entire page, so that it looks to an unsuspecting user like he or she is actually visiting example.com. Within the malicious site the attacker often embeds javascript, such as a keylogger, which listens for all key events on the page in an effort to steal data from the unsuspecting user. A page vulnerable to XFS also leads to other exploits such as Clickjacking, Cross-Site Scripting, and Session Hijacking. Clickjacking for example takes the form of embedded code or script that can execute without the user's knowledge, such as clicking on a button that appears to perform another function. Cross-Frame Scripting is often preceded by a phishing attack such as by sending a malicious hyperlink to a user and enticing them to click on it. The link will appear to take the user to a legitimate site (example.com) however, an attacker controlled site is loaded acting as a man-in-the-middle between the client and legitimate site which has been rendered in the malicious iframe.
For more information, please visit: https://www.owasp.org/index.php/Cross_Frame_Scripting https://www.owasp.org/index.php/Clickjacking

Severity Description:

An attacker may be able to frame a legitimate application inside a malicious frame which can in turn be used as a man in the middle, capture keystrokes or any number of other malicious actions. All activity by the user can be monitored and recorded by the attacker allowing compromise of usernames, passwords, session data or any other potentially sensitive data input by the user.

Vulnerability Severity:

- **Risk -** Medium
- **Impact -** Medium
- **Exploitability -** Medium
- **CVSS Score: -** CVSS v3.1 Vector: AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N = 4.2 **(Medium)**

Category: Client-Side Attack

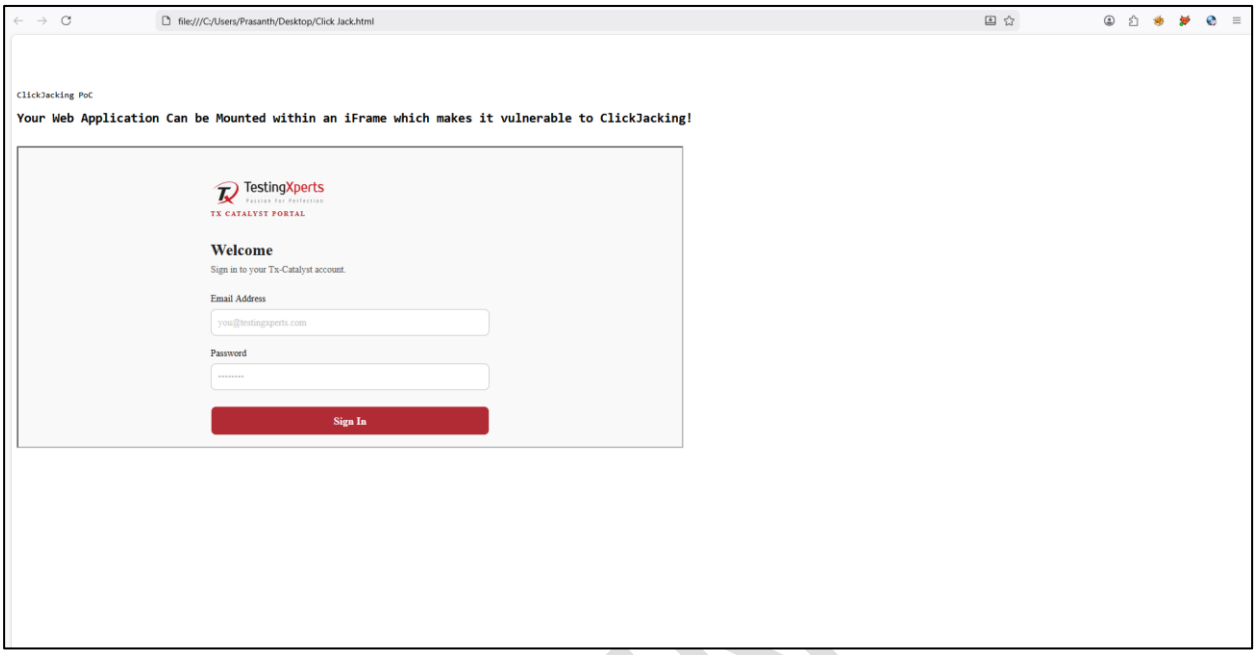
Instance:

http://192.168.3.90:3002/

Proof-Of-Concept:

Provided are the screenshot and Clickjacking PoC which demonstrate that the application is vulnerable to clickjacking.

Screenshot:



Clickjacking PoC:

```
HTML Script:
<pre lang="JavaScript" line="1">
<html>
<head>
<title>ClickJacking PoC</title>
</head>
ClickJacking PoC
<h2>Your Web Application Can be Mounted within an iFrame which makes it
vulnerable to ClickJacking!</h2>
<iframe          src="http://192.168.3.90:3002/login"          height="450"
width="1000"></iframe>
</body>
</html>
</pre>
```

Remediation:

Return an appropriate "Content-Security-Policy" and "X-Frame-Options" header as outlined in the following guidelines to prevent malicious framing attacks: https://www.owasp.org/index.php/Clickjacking_Defense_Cheat_Sheet
https://www.owasp.org/index.php/Cross_Frame_Scripting
These include suggestions such as: Content-Security-Policy: frame-ancestors a-trusted-site.com (as well as 'self' and 'none' in place of the example domain) X-Frame-Options: ALLOW-FROM a-trusted-site.com (as well as "DENY" and "SAMEORIGIN" options) Please note that while the "X-Frame-Options" header was always considered experimental and is superseded by Content-Security-Policy definitions, our recommendation is to return both headers, to ensure backwards compatibility with older web browsers.

Low Severity Vulnerabilities

2.5 Missing Cookie 'Flags'

Vulnerability Information:

The application sets cookies without enforcing appropriate security attributes such as 'Secure', 'HttpOnly', and 'SameSite'. The absence of these attributes increases the risk of cookies being accessed via client-side scripts, transmitted over unencrypted channels, or included in cross-site requests.

Exploit Description:

An attacker may exploit missing cookie attributes by intercepting or manipulating cookies during transmission or execution within the browser. Without the 'Secure' flag, cookies can be transmitted over unencrypted HTTP connections, making them susceptible to interception via man-in-the-middle attacks. In the absence of the 'HttpOnly' attribute, cookies can be accessed through client-side scripts, enabling attackers to steal session identifiers through cross-site scripting attacks. Additionally, without the 'SameSite' attribute, cookies may be included in cross-origin requests, increasing the risk of cross-site request forgery attacks.

Severity Description:

Exploitation of missing cookie security attributes may result in unauthorized access to user sessions, leakage of sensitive information, and potential account compromise. Attackers may leverage these weaknesses to hijack authenticated sessions or perform actions on behalf of legitimate users.

Vulnerability Severity:

- **Risk -** Low
- **Impact -** Low
- **Exploitability -** Low
- **CVSS Score: -** CVSS v3.1 Vector: AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N = 3.1 **(Low)**

Category: Session Management

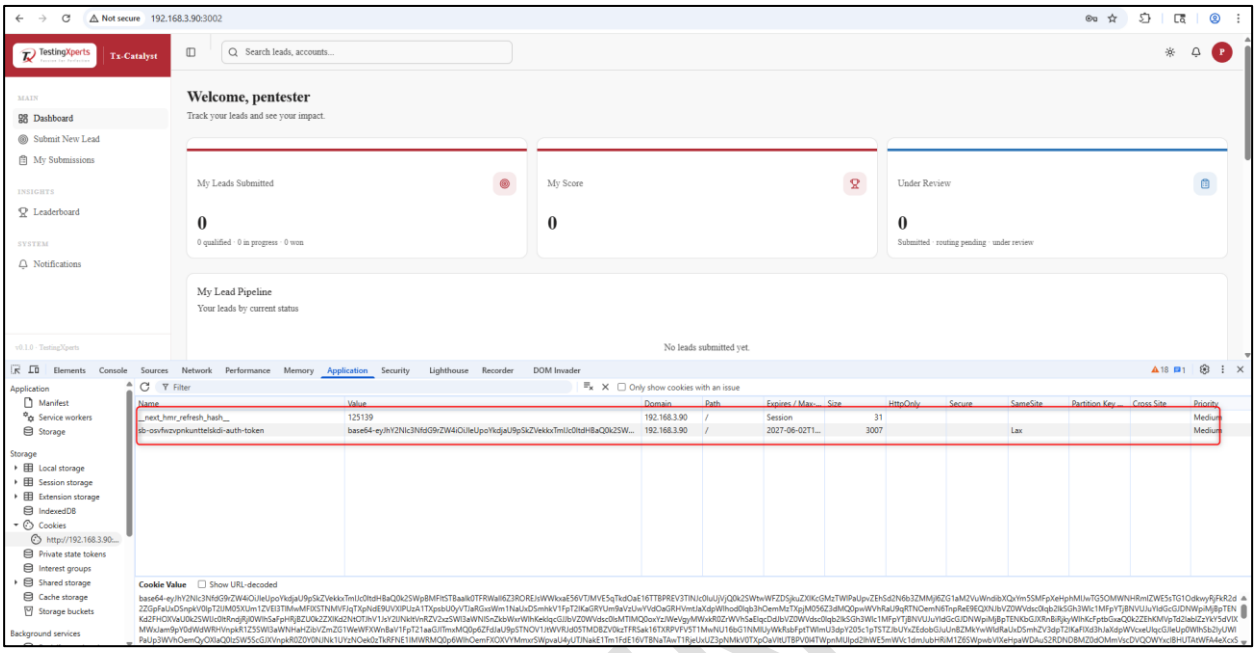
Instance:

http://192.168.3.90:3002/

Proof-Of-Concept:

Provided is the screenshot which demonstrate that the application is missing cookies attributes.

Screenshot:



Remediation:

Ensure that all sensitive cookies are configured with the appropriate security attributes as outlined in industry best practices. This includes:

- Setting the 'Secure' flag to ensure cookies are only transmitted over HTTPS connections.
- Setting the 'HttpOnly' flag to prevent client-side scripts from accessing cookie values.
- Implementing the 'SameSite' attribute (e.g., Strict or Lax) to mitigate cross-site request forgery risks.

2.6 Missing Security Headers

Vulnerability Information:

The application does not implement several recommended HTTP security headers, including Content-Security-Policy (CSP), X-Frame-Options, Strict-Transport-Security (HSTS), X-Content-Type-Options, and X-XSS-Protection. The absence of these headers reduces the browser's ability to enforce security controls, thereby increasing exposure to multiple client-side attacks.

Exploit Description:

In the absence of security headers, attackers may exploit the application through various client-side attack vectors. Without CSP, malicious scripts may be executed in the browser. Missing X-Frame-Options allows the application to be embedded within iframes, enabling clickjacking attacks. The lack of HSTS permits downgrade attacks to insecure HTTP connections. Without X-Content-Type-Options, browsers may incorrectly interpret content types, increasing the risk of

MIME sniffing attacks. Additionally, absence of X-XSS-Protection reduces protection against reflected XSS in legacy browsers.

Severity Description:

Exploitation of missing security headers may lead to multiple attack scenarios, including cross-site scripting, clickjacking, data interception, and content injection. This can result in compromise of user sessions, exposure of sensitive information, and unauthorized actions performed on behalf of users.

Vulnerability Severity:

- **Risk -** Medium
- **Impact -** Low
- **Exploitability -** Low
- **CVSS Score: -** CVSS v3.1 Vector: AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N = 3.1 **(Low)**

Category: Configuration

Instance:

http://192.168.3.90:3002/

Proof-Of-Concept:

Provided are the request and response demonstrating that HTTP security headers are missing in the application.

Vulnerable Request/Response:

Request:

```
GET / HTTP/1.1
Host: 192.168.3.90:3002
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/web
p,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
[Redacted for Brevity]
```

Response:

```
HTTP/1.1 200 OK
Vary: rsc, next-router-state-tree, next-router-prefetch, next-router-
segment-prefetch, Accept-Encoding
link: </_next/static/media/fba5a26ea33df6a3-s.p.1bbdebe6.woff2>;
rel=preload; as="font"; crossorigin=""; type="font/woff2"
```

```
Cache-Control: no-store, must-revalidate
X-Powered-By: Next.js
Content-Type: text/html; charset=utf-8
Date: Tue, 28 Apr 2026 09:50:09 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 53795
[Redacted for Brevity]
```

Remediation:

Ensure that all recommended security headers are properly configured and returned in server responses. These include:

- Content-Security-Policy (CSP) to restrict trusted content sources and mitigate XSS attacks.
- X-Frame-Options or CSP frame-ancestors directive to prevent clickjacking attacks.
- Strict-Transport-Security (HSTS) to enforce HTTPS communication.
- X-Content-Type-Options: nosniff to prevent MIME type sniffing.
- X-XSS-Protection to enable basic XSS filtering in legacy browsers.

Informational Vulnerabilities

2.7 Lack of Post-Quantum Cryptography (PQC) Support

Vulnerability Information:

The application does not fully support modern Post-Quantum Cryptography (PQC) algorithms as recommended by NIST, which may pose a risk against future quantum computing attacks. Current implementation primarily relies on classical cryptographic algorithms (e.g., RSA/ECDSA), which are considered vulnerable to “Harvest Now, Decrypt Later” attacks in the long term.

Exploit Description:

An attacker with the capability to capture encrypted traffic today may store it and decrypt it in the future once quantum computing becomes viable, due to the absence of strong PQC mechanisms.

Severity Description:

This could result in long-term exposure of sensitive information such as authentication tokens, session data, and confidential communications.

Vulnerability Severity: Informational

Category: Cryptography

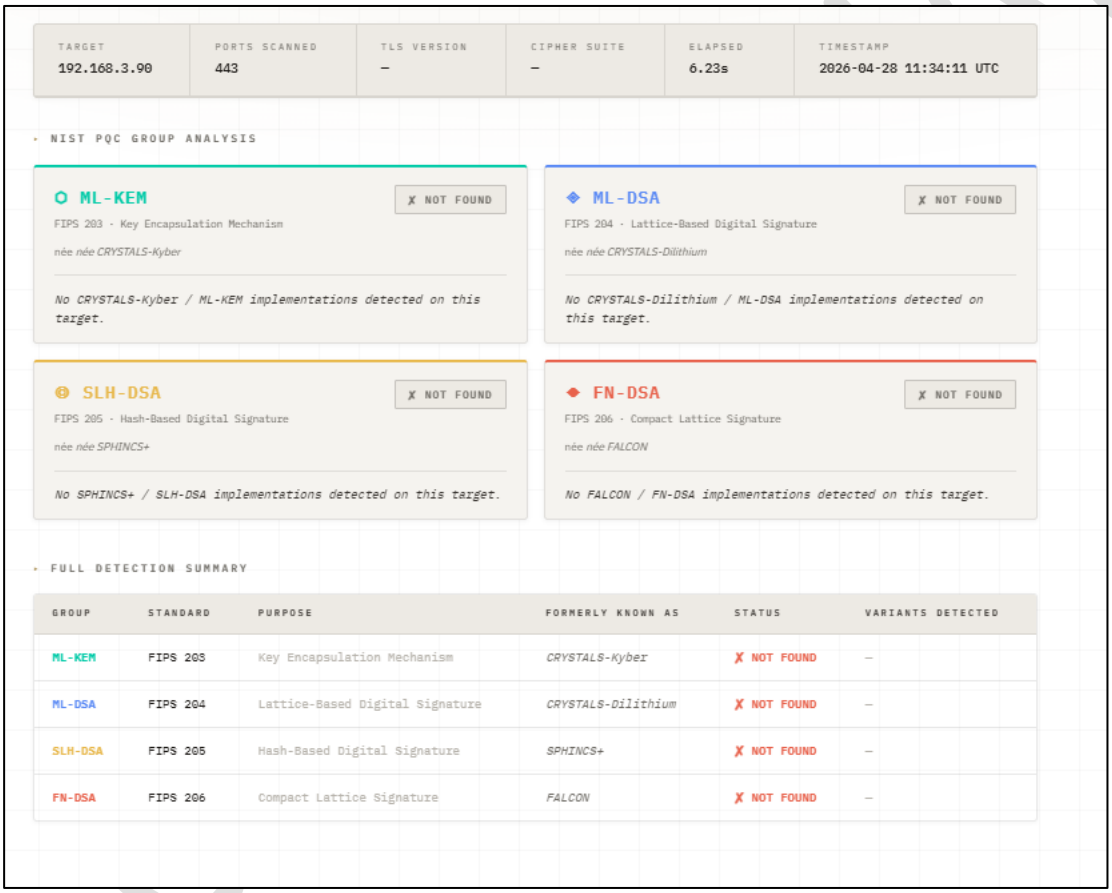
Instance:

http://192.168.3.90:3002/

Proof-Of-Concept:

Provided are the outputs from the Post-Quantum Cryptography detection tool, which demonstrate that the application is not fully quantum-ready and lacks support for multiple NIST-recommended PQC algorithms.

Screenshot:



Remediation:

1. Implement NIST-approved Post-Quantum Cryptographic algorithms such as:
    a. ML-KEM (Key Exchange)
    b. ML-DSA / SLH-DSA / FN-DSA (Digital Signatures)
2. Adopt hybrid cryptographic models combining classical + PQC algorithms.
3. Ensure TLS configurations support PQC-enabled cipher suites.

4. Continuously monitor NIST PQC standardization updates and upgrade cryptographic implementations accordingly.

2.8 Weak Cryptography

Vulnerability Information:

The application uses a non-cryptographic pseudorandom number generator (Math.random()) to generate values. Such PRNGs are not designed to be unpredictable and can produce deterministic outputs under certain conditions. This makes them unsuitable for any functionality requiring secure randomness.

Exploit Description:

If the generated values are used in security-sensitive contexts (e.g., session identifiers, tokens, password reset links, or authentication mechanisms), an attacker may be able to predict or reproduce these values. By exploiting this predictability, the attacker could guess valid tokens or identifiers and gain unauthorized access or impersonate legitimate users.

Severity Description:

The impact depends entirely on where the weak randomness is used. If tied to authentication or sensitive operations, this could lead to account takeover, session hijacking, or unauthorized data access. However, if the randomness is used only for non-security purposes (e.g., UI rendering or visual variation), the issue has no real security impact and should be treated as a false positive.

Vulnerability Severity: Informational

Category: Cryptography

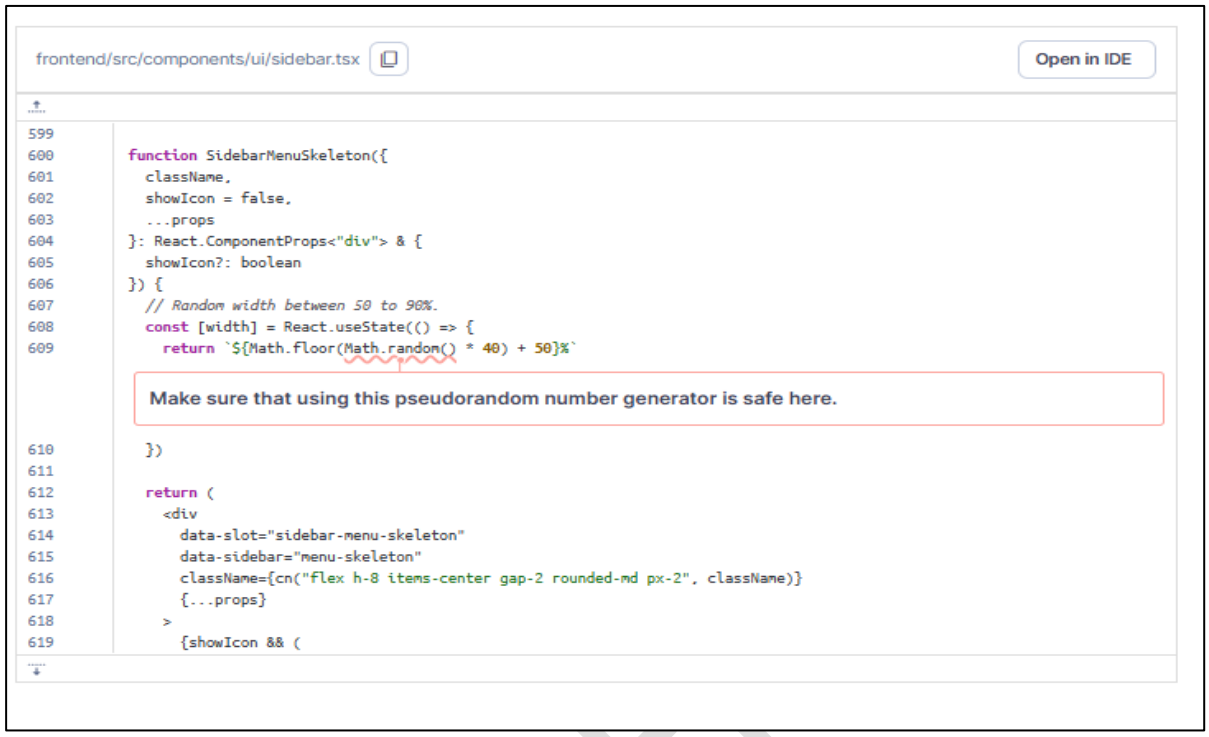
Instance:

http://192.168.3.90:3002/

Proof-Of-Concept:

Provided is the screenshot which shows that the application has weak cryptography.

Screenshot:



Remediation:

Use a cryptographically secure random generator (e.g., `crypto.getRandomValues()` or `crypto.randomBytes()`) instead of `Math.random()` for any security-sensitive value generation.

3. TestingXperts Methodology

Software Application Security Testing (SAST) and Software Composition Analysis (SCA) is conducted to observe the application code in a run-time environment and to simulate real-world attack scenarios. Automated testing can identify design flaws, evaluate environmental conditions, compound multiple lower risk flaws into higher risk vulnerabilities, and determine if identified flaws affect the confidentiality, integrity, or availability of the application.

Objectives:-
The stated objectives of a SAST and SCA are:

- Perform testing, using proprietary and/or public tools, to determine whether it is possible for an attacker to:
    - Circumvent authentication and authorization mechanisms
    - Escalate application user privileges
    - Hijack accounts belonging to other users
    - Violate access controls placed by the site administrator
    - Alter data or data presentation
    - Corrupt application and data integrity, functionality and performance
    - Circumvent application business logic
    - Circumvent application session management
    - Break or analyse use of cryptography within user accessible components
    - Determine possible extent access or impact to the system by attempting to exploit vulnerabilities
- Score vulnerabilities using the Common Vulnerability Scoring System (CVSS v3.1)
- Provide tactical recommendations to address security issues of immediate consequence
- Provide strategic recommendations to enhance security by leveraging industry best practices Attack vectors

In order to achieve the stated objectives, the following tests are performed as part of the automated and manual vulnerability assessment and penetration testing, when applicable to the platforms and technologies in use:

- Cross Site Scripting (XSS)
- SQL Injection
- Command Injection
- Cross Site Request Forgery (CSRF)
- Authentication/Authorization Bypass
- Session Management testing, e.g. token analysis, session expiration, and logout effectiveness
- Account Management testing, e.g. password strength, password reset, account lockout, etc.

- Directory Traversal
- Response Splitting
- Stack/Heap Overflows
- Format String Attacks
- Cookie Analysis
- Server Side Includes Injection
- Remote File Inclusion
- LDAP Injection
- XPATH Injection
- Internationalization attacks
- Denial of Service testing at the application layer only
- AJAX Endpoint Analysis
- Web Services Endpoint Analysis
- HTTP Method Analysis
- SSL Certificate and Cipher Strength Analysis
- Forced Browsing

Risk Rating Methodology

TestingXperts uses a CVSS V 3.1 calculator to determine the severity of the identified vulnerabilities. We take into account the attack complexity, attack vector, level of privileges, user interaction and the impact on Confidentiality, Integrity and Availability of the application to determine the severity of the vulnerability.

The CVSS assessment measures three areas of concern:

- Base Metrics for qualities intrinsic to a vulnerability
- Temporal Metrics for characteristics that evolve over the lifetime of vulnerability
- Environmental Metrics for vulnerabilities that depend on an implementation or environment

Along with the CVSS score we provide Risk Rating for the vulnerabilities as well, the Risk Rating is TestingXperts recommended prioritization for addressing findings.

Based on the severity of the vulnerability, they are assigned below ratings:

CVSS Severity Rating	CVSS Score
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Informational	0.0

Overall Risk

Overall risk reflects TestingXperts estimation of the risk that a finding poses to the target system or systems have. It takes into account the impact of the finding, the difficulty of exploitation, and any other relevant factors.

Critical -Implies an immediate, easily accessible threat of total compromise.
High- Implies an immediate threat of system compromise, or an easily accessible threat of large-scale breach.
Medium- A difficult to exploit threat of large-scale breach, or easy compromise of a small portion of the application.
Low- Implies a relatively minor threat to the application.
Informational- No immediate threat to the application. May provide suggestions for application improvement, functional issues with the application, or conditions that could later lead to an exploitable finding.

Impact

Impact reflects the effects that successful exploitation upon the target system or systems. It takes into account potential losses of confidentiality, integrity and availability, as well as potential reputational losses.

High - Attackers can read or modify all data in a system, execute arbitrary code on the system, or escalate their privileges to superuser level.
Medium - Attackers can read or modify some unauthorized data on a system, deny access to that system, or gain significant internal technical information.
Low - Attackers can gain small amounts of unauthorized information or slightly degrade system performance. May have a negative public perception of security.

Exploitability

Exploitability reflects the ease with which attackers may exploit a finding. It takes into account the level of access required, availability of exploitation information, requirements relating to social engineering, race conditions, brute forcing, etc, and other impediments to exploitation.

High - Attackers can unilaterally exploit the finding without special permissions or significant roadblocks.
Medium- Attackers would need to leverage a third party, gain non-public information, exploit a race
condition, already have privileged access, or otherwise overcome moderate hurdles in order to exploit the finding.
Low- Exploitation requires implausible social engineering, a difficult race condition, guessing difficult-to guess data, or is otherwise unlikely.

Category

TestingXperts categorizes findings based on the security area to which those findings belong. This can help organizations identify gaps in secure development, deployment, patching, etc.

- **Authorization** - Related to authorization of users, and assessment of rights.
- **Auditing and Logging** - Related to auditing of actions, or logging of problems.
- **Authentication** - Related to the identification of users.
- **Configuration** - Related to security configurations of servers, devices, or software.
- **Cryptography** - Related to mathematical protections for data.
- **Data Exposure** - Related to unintended exposure of sensitive information.
- **Input Validation** - Related to improper reliance on the structure or values of data.
- **Denial of Service** - Related to causing system failure.
- **Error Reporting** - Related to the reporting of error conditions in a secure fashion.
- **Patching** - Related to keeping software up to date.
- **Session Management** - Related to the session management schema implemented in the application.
- **Timing** - Related to race conditions, locking, or order of operations.
- **Business Logic Bypass** – Related to bypassing the business logic implemented by the application.
- **Client Side Attack** – Related to execution of the code on the client, typically natively within a web browser or browser plugin

4. Conclusion

It has been observed that the application is vulnerable to publicly known attacks. We recommend mitigating the high-risk vulnerabilities on priority, followed by medium risk vulnerabilities, low-risk vulnerabilities and then informational vulnerabilities. Additionally, a full cycle of security testing must be carried out periodically or before a major release of the application.